10 · Exp 11 — Singly linked list (insert, delete, traverse)

The pointers from Exp 1 finally pay off. New C++: struct, new / delete, and the arrow operator ->. Read 10.1 slowly; everything after builds on it.

struct Node → new Node → head → insert at end → delete from beginning → traverse until NULL

10.1 New C++: struct, new, delete, ->

A **struct** bundles several variables into one new type:

```
struct Node
{
    int data;      // the value
    Node *next;    // address of the next node (NULL if none)
};
```

- `new Node` asks for memory for one Node **while the program runs** and gives back its **address**: `Node *newNode = new Node;`
- `p->data` means "the `data` field of the node that `p` points to". It is shorthand for `(*p).data`.
- `delete p;` gives that node's memory back. Every `new` should have a matching `delete`.
- `NULL` is the "points nowhere" address: the end of the list, or an empty list (`head == NULL`).

GUESS FIRST

An array keeps its elements side by side. How does a linked list know where its second element is, if the nodes can be anywhere in memory?

HEAD → 10 → 20 → 30 → NULL



Insert 40 at end: walk temp to the node whose next is NULL, then

```
temp->next = newNode;
```

Delete from beginning:

```
temp = head; head = head->next; delete temp;
```

Order matters: move head BEFORE delete, or the rest of the list is lost.

The manual's list after inserting 10, 20, 30. Each node stores the address of the next.

The list is reached only through `head`, the address of the first node. Lose `head` and the whole list is lost.

10.2 The three operations

Operation	Steps	Time
insert at end	make node (next = NULL) → if `head == NULL`, head = node → else walk `temp` until `temp->next == NULL`, then `temp->next = newNode`	O(n) (the walk)
delete from beginning	if empty → message → else `temp = head`, `head = head->next`, `delete temp`	O(1)
traverse	`temp = head`; while `temp != NULL`: print `temp->data`, `temp = temp->next`	O(n)

THE TWO CLASSIC LINKED-LIST BUGS

1. **Forgetting `temp = temp->next;`** in traverse: `temp` never moves, so the loop never ends. The manual's IMPLEMENTATION screenshot (page 9) has exactly this bug, and its insert never links the new node in. Its **listing** (page 6) is correct; use that.
2. **`delete head;` before moving head**: the second node's address was inside the deleted node, so the rest of the list is lost. Always save the node in `temp`, move `head`, *then* `delete temp`.

10.3 The program

The manual's listing was correct (one line was cut off at the page edge: `cout << temp->data << " deleted from list." << endl;`).

E11_SINGLY_LINKED_LIST.CPP

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;                          // empty list

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;               // ask for memory for one node
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != NULL)          // walk to the last node
            temp = temp->next;
```

```
        temp->next = newNode;
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << temp->data << " deleted from list." << endl;
        head = head->next;                  // move head first...
        delete temp;                        // ...then free the old first node
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Linked List: ";
        while (temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;              // without this line: infinite loop
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
```

```
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

The manual's Sample Inputs 1 and 2 in one run:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Edge cases (delete and traverse on an empty list, delete the only node):

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 2
List is empty.
Enter your choice: 3
List is empty.
Enter your choice: 1
Enter value to insert: 5
5 inserted into list.
Enter your choice: 2
```

```
5 deleted from list.
Enter your choice: 3
List is empty.
Enter your choice: 4
Program terminated.
```

WRITE IN YOUR RECORD

Aim: To implement a singly linked list and perform insertion, deletion and traversal.

Algorithm: head = NULL. *Insert (end):* create node (data = value, next = NULL); if head == NULL, head = node; else move temp to the last node (temp->next == NULL) and set temp->next = node. *Delete (beginning):* if head == NULL → "empty"; else temp = head, head = head->next, delete temp. *Traverse:* temp = head; while temp ≠ NULL print temp->data, temp = temp->next.

Complexity: insert at end O(n), delete at beginning O(1), traverse O(n); space O(n) for n nodes.

Result: Inserted 10, 20, 30 (list: 10 20 30); deleting from the beginning removed 10 (list: 20 30).

10.4 Viva questions

Question	Answer
Linked list vs. array?	List: nodes anywhere in memory, size grows/shrinks, insert/delete by changing pointers, but no direct `a[i]` access (must walk) and an extra pointer per node. Array: contiguous, fixed size, O(1) indexing, O(n) insert/delete.
What does the last node's next hold?	NULL.
What is head?	A pointer to the first node; `head == NULL` means the list is empty.
Why is insert at end O(n)?	You must walk from head to the last node. (Keeping a `tail` pointer makes it O(1).)
Why is delete at beginning O(1)?	Only head changes; no walking.
What is `->`?	Access a field through a pointer: `p->data` = `(*p).data`.
Why `delete temp`?	Return the node's memory; otherwise it leaks.
Can you traverse backwards?	Not in a singly linked list (only next pointers). A doubly linked list adds `prev`.

10.5 If the examiner changes the question

Insert at end and delete from beginning are the experiment. The rest (9 checks, all pass), each a function to add above `main()` and to the menu:

Insert at the beginning (O(1)):

```
void insertBegin(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;                    // new node points to the old first node
    head = newNode;                          // and becomes the first node
}
```

Delete from the end. Stop at the second-last node:

```
void deleteEnd()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
        return;
    }
    if (head->next == NULL)                  // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
        return;
    }
    Node *temp = head;
    while (temp->next->next != NULL)         // stop at the SECOND-last node
        temp = temp->next;
    cout << temp->next->data << " deleted from list." << endl;
    delete temp->next;
    temp->next = NULL;                       // it is now the last node
}
```

Search (returns the position) and **count nodes**:

```
int search(int key)                          // 1-based position, -1 if absent
{
    int pos = 1;
    Node *temp = head;
    while (temp != NULL)
    {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    }
    return -1;
}
```

```
int countNodes()
{
    int count = 0;
    for (Node *temp = head; temp != NULL; temp = temp->next)
        count++;
    return count;
}
```

Display in reverse (recursion: the deepest call prints first):

```
// Recursion: first print the REST of the list in reverse, then this node
void displayReverse(Node *p)
{
    if (p == NULL)
        return;
    displayReverse(p->next);
    cout << p->data << " ";
}
// call it as:  displayReverse(head);
```

Insert at a position:

```
void insertAtPos(int pos, int value)         // pos 1 = new first node
{
    if (pos < 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)
    {
        insertBegin(value);
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)   // go to node pos-1
        temp = temp->next;
    if (temp == NULL)
    {
        cout << "Invalid position." << endl;
        return;
    }
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;              // 1) new node points ahead
    temp->next = newNode;                    // 2) then link it in (order matters!)
}
```

Delete at a position:

```
void deleteAtPos(int pos)
{
    if (head == NULL || pos < 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    Node *del;
    if (pos == 1)
    {
        del = head;
        head = head->next;
    }
    else
    {
        Node *temp = head;
        for (int i = 1; i < pos - 1 && temp != NULL; i++)   // go to node pos-1
            temp = temp->next;
```

```
        if (temp == NULL || temp->next == NULL)
        {
            cout << "Invalid position." << endl;
            return;
        }
        del = temp->next;
        temp->next = del->next;              // bypass the node
    }
    cout << del->data << " deleted from list." << endl;
    delete del;
}
```

CHECK 10.1

For inserting a new node after node `temp`, these two lines come in some order: `temp->next = newNode;` and `newNode->next = temp->next;`. Which must come first, and what happens if you swap them?

CHECK 10.2

List: head → 4 → 8 → NULL. Trace `deleteNode()` then `insert(5)`. Draw the list after each.

Answers

Check 10.1: `newNode->next = temp->next;` **first**. If `temp->next = newNode;` runs first, the old `temp->next` address is overwritten, so `newNode->next` would point to newNode itself and the rest of the list is lost.

Check 10.2: delete: temp = (4), head = (8), delete temp → head → 8 → NULL. insert(5): walk to 8 (next is NULL), link → head → 8 → 5 → NULL.